

RDC News Intelligence — Chapitre IV

Déploiement

Chapitre 4 — Déploiement de l'application RDC News Intelligence

1. Introduction
2. Vue de déploiement
3. Prérequis matériels
4. Configuration d'exécution
5. Procédure de déploiement
6. Migration des bases de données
7. Exploitation et surveillance
8. Contraintes et risques
9. Canaux de déploiement messagerie
10. Multilinguisme en déploiement
11. Évolutions de déploiement
12. Conclusion du chapitre
13. Figures et documents associés

Chapitre 4 — Déploiement de l'application RDC News Intelligence

Attribut	Valeur
Document	Chapitre_4_Deploiement
Projet	RDC News Intelligence
Version	1.0
Prérequis	Chapitre 2 (README_CHAPITRE_2.md) · Chapitre 3 (Chapitre_3_Modelisation.md)

1. Introduction

1.1 Objet du chapitre

Le chapitre 2 a décrit l'architecture logique (Orchestrateur, Engine, bases de données). Le chapitre 3 en a donné la modélisation (cas d'utilisation, séquences, données). Le présent chapitre traite du **déploiement** : environnement cible, stack logicielle, configuration, migration des données et contraintes d'exploitation.

1.2 Stratégie retenue : un serveur unique (VPS)

L'architecture **cible** pour la production regroupe sur **une même machine virtuelle** (VPS) :

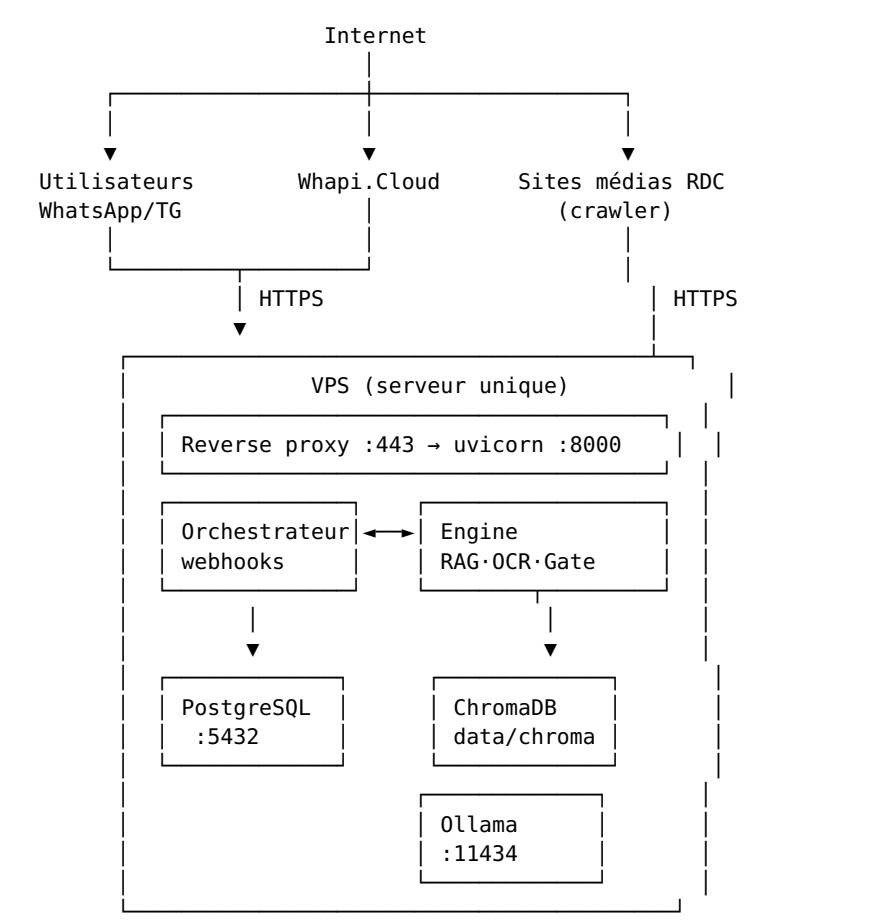
- l'application FastAPI (ai-service) ;
- PostgreSQL ;
- ChromaDB (fichiers locaux data/chroma_db) ;
- Ollama (Mistral quantifié) ;
- le reverse proxy HTTPS (nginx ou OpenLiteSpeed).

Les blocs **Orchestrateur** et **Engine** restent **logiquement séparés** dans le code ; physiquement ils partagent le même processus et les mêmes ressources. Les échanges queue/pop et reply-relay utilisent de préférence **http://127.0.0.1:8000** pour éviter les timeouts du proxy externe.

Note : un prototype « VPS léger + PC local pour l'IA » a existé pendant le développement ; la cible documentée ici est la **consolidation sur VPS** pour la soutenance et l'exploitation continue.

2. Vue de déploiement

2.1 Diagramme (niveau infrastructure)



2.2 Composants logiciels

Composant	Version / type	Rôle
OS	Linux (Ubuntu/Debian)	Hébergement

Python	3.10+	Runtime ai-service
FastAPI + Uvicorn	—	API REST, webhooks
PostgreSQL	14+ (pgvector optionnel)	Catalogue articles
ChromaDB	Client persistant	Index vectoriel
Ollama	—	Inférence Mistral local
Tesseract	+ langues fra, eng	OCR images WhatsApp
Reverse proxy	nginx / OpenLiteSpeed	TLS, routage /webhooks, /health

3. Prérequis matériels

Ressource	Minimum recommandé	Commentaire
RAM	8 Go	Mistral 7B Q4 + embeddings + Postgres
CPU	4 vCPU	Crawler, encoding, Ollama
Disque	40–80 Go SSD	Corpus, chroma_db, modèles Ollama, logs
Réseau	IP publique, ports 443/80	Webhooks Whapi/Telegram

En dessous de 8 Go RAM, Ollama peut échouer (requires more system memory) ou répondre très lentement (plusieurs minutes par message).

4. Configuration d'exécution

4.1 Variables d'environnement (profil production VPS)

Fichier typique : ai-service/.env ou .env_file (non versionné).

Messagerie Whapi

Variable	Exemple
WHAPI_TOKEN	(secret)

WHAPI_WEBHOOK_PROXY_ONLY	true
WHAPI_QUEUE_TOKEN	secret partagé
WHAPI_REPLY_RELAY_TOKEN	secret partagé
WHAPI_QUEUE_POP_URL	http://127.0.0.1:8000/webhooks/whapi/que
WHAPI_REPLY_RELAY_URL	http://127.0.0.1:8000/webhooks/whapi/ref relay
ENABLE_WHAPI_QUEUE_POLLING	true

Base de données

Variable	Exemple
DATABASE_URL	postgresql://user:pass@127.0.0.1:5432/rdc_news

Intelligence artificielle

Variable	Exemple
OLLAMA_HOST	http://127.0.0.1:11434
OLLAMA_MODEL	mistral:7b-instruct-v0.3-q4_K_M
RAG_MIN_SIMILARITY_MSG	0.40
WHATSAPP_TOP_K	3
RAG_ENABLE_RERANK	true ou false
TOPIC_GATE_MIN_CONFIDENCE	0.6

Telegram (optionnel)

Variable	Exemple
TELEGRAM_BOT_TOKEN	(BotFather)
ENABLE_TELEGRAM_POLLING	true (sans webhook)

Crawler / maintenance

Variable	Exemple
ENABLE_CRON_JOBS	true

4.2 Endpoints publics exposés

Chemin	Usage
POST /webhooks/whapi	Webhook Whapi (HTTPS public)
POST /webhooks/telegram	Webhook Telegram (si utilisé)

GET /health	Sonde disponibilité
GET /admin/overview	Statistiques (protéger en prod)

Les routes queue/pop et reply-relay sont appelées en **interne** ; elles n'ont pas besoin d'être exposées au domaine public si tout tourne sur le même hôte.

5. Procédure de déploiement

5.1 Installation (ordre recommandé)

1. Cloner le dépôt sur le VPS.
2. Créer un environnement virtuel Python ; `pip install -r requirements.txt`.
3. Installer et démarrer **PostgreSQL** ; créer la base `rdc_news`.
4. Installer **Ollama** ; `ollama pull mistral:7b-instruct-v0.3-q4_K_M` (ou modèle choisi).
5. Installer **Tesseract** (`tesseract-ocr`, `tesseract-ocr-fra`, `tesseract-ocr-eng`).
6. Copier le fichier `.env` avec les variables §4.1.
7. Migrer les données (§6).
8. Lancer l'application : `uvicorn app.main:app --host 0.0.0.0 --port 8000` ou **PM2** / `systemd`.
9. Configurer le **reverse proxy** vers le port 8000.
10. Enregistrer l'URL webhook chez **Whapi** : `https://<domaine>/webhooks/whapi`.

5.2 Processus de gestion (exemple PM2)

Processus	Commande type
API	<code>uvicorn app.main:app --host 127.0.0.1 --port 8000</code>
Ollama	<code>service système ollama</code>

Le polling Whapi et le polling Telegram sont des **tâches asyncio** démarrées au startup de FastAPI si les flags `ENABLE_*` sont actifs.

5.3 Crawler en production

```
cd ai-service
python -m app.services.crawler.scripts.sync --source-id all
```

Planification : `cron` ou `ENABLE_CRON_JOBS=true` dans l'application.

6. Migration des bases de données

6.1 PostgreSQL (machine de dev → VPS)

Export (poste source) :

```
pg_dump -h localhost -U USER -d DB_NAME -Fc -f rdc_articles.dump
```

Transfert et import (VPS) :

```
scp rdc_articles.dump user@vps:/tmp/  
pg_restore -h 127.0.0.1 -U user_vps -d rdc_news -Fc  
/tmp/rdc_articles.dump
```

6.2 ChromaDB

Option A — copie du dossier (rapide) :

```
scp -r ai-service/data/chroma_db user@vps:/path/to/ai-service/data/
```

Option B — reconstruction depuis Postgres :

```
cd ai-service  
python scripts/sync_to_chroma.py
```

Vérifier l'alignement : GET /admin/overview → comparer total_articles et le compteur Chroma.

6.3 Vérifications post-migration

Contrôle	Commande / action
API vivante	curl http://127.0.0.1:8000/health
Postgres	SELECT COUNT(*) FROM articles;
Chroma	logs [VectorStoreService] + admin overview
Ollama	curl http://127.0.0.1:11434/api/tags
Whapi	message test en privé puis en groupe

7. Exploitation et surveillance

7.1 Journaux

Fichier / flux	Contenu
.logs/fastapi.log	Requêtes, RAG, erreurs webhook
Logs Uvicorn / PM2	Démarrage, crash
training_runs (SQL)	Historique sync Chroma

7.2 Indicateurs utiles

Indicateur	Source
Nombre d'articles	/admin/overview
Écart Postgres vs Chroma	même endpoint
Latence réponse	différence timestamp webhook → relay
Erreurs Ollama	logs LLMService, mémoire insuffisante

7.3 Sauvegarde

Élément	Fréquence suggérée
	quotidien

Dump PostgreSQL

Copie data/chroma_db hebdomadaire ou après gros crawl

Fichier .env hors dépôt git, sauvegarde chiffrée

8. Contraintes et risques

Risque	Impact	Atténuation
RAM insuffisante	Ollama 500, réponses absentes	VPS 8 Go+, modèle Q4, RAG_ENABLE_RERANK=false
Webhook indisponible	Pas de nouveaux messages	Monitoring /health, HTTPS valide
Désalignement Chroma/Postgres	Retrieval incomplet	sync_to_chroma, training_runs
Timeout Whapi	File qui grossit	Réponse 200 rapide, worker polling actif
Corpus uniquement FR	SW/EN partiels	Documenter limite linguistique (article)

9. Canaux de déploiement messagerie

9.1 WhatsApp via Whapi

Mode	Configuration
Production recommandée	Webhook → VPS ; polling + relay en 127.0.0.1
Privé 1:1	Supporté ; Topic Gate désactivé
Groupe	Supporté ; Topic Gate activé

9.2 Telegram

Mode	Configuration
Polling	ENABLE_TELEGRAM_POLLING=true, pas de HTTPS requis
Webhook	POST /webhooks/telegram + setWebhook

10. Multilinguisme en déploiement

Langue	Entrée utilisateur	Corpus déployé	Commentaire exploitation
Français	Complet	Complet (médias RDC FR)	Configuration nominale
Anglais	Complet	Partiel (selon articles)	Modèle embedding

		indexés)	multilingue
Swahili	Technique possible	Insuffisant aujourd'hui	Ajouter sources SW au crawl avant promesse produit

Le paramétrage linguistique ne nécessite pas de service séparé : il dépend du **contenu de articles** et du modèle d'embedding partagé.

11. Évolutions de déploiement

Évolution	Bénéfice
Postgres managé (RDS, etc.)	Sauvegardes, montée en charge SQL
Chroma serveur distant	Séparation stockage (optionnel)
GPU sur VPS	Accélération Ollama
Frontend admin derrière auth	Sécuriser /admin/overview
Tables verifications	Audit et anti-surinformation persistée

12. Conclusion du chapitre

Le déploiement de RDC News Intelligence repose sur un **VPS unique** regroupant proxy HTTPS, FastAPI (Orchestrateur + Engine), PostgreSQL, ChromaDB et Ollama. La configuration critique est portée par les variables Whapi (webhook, file, relay interne), DATABASE_URL et OLLAMA_HOST. La migration du corpus s'effectue par pg_dump/pg_restore et copie ou régénération de data/chroma_db.

Avec les chapitres 2 (architecture), 3 (modélisation) et 4 (déploiement), le lecteur dispose d'une chaîne complète : **conception** → **structure** → **mise en production**.

13. Figures et documents associés

Document	Lien
Architecture	architecture-memoire-duale- rdc-news.png
Bases de données	architecture-bases-donnees- rdc-news.png
Guide technique	README.md
Flux Whapi	FLUX_WHATSAPP_VERS_DRAWIO.md

Fin du Chapitre 4 — Déploiement.